Software Requirements Specification

for

AI-Driven Detection of Data-Exfiltration Malware on Mobile Devices

Version 1.0 approved

Prepared by Bentley Group 6
Victor Gan
Tobias Jun
Atticus Kong
Mark Pham
Jacob Sepkus
Nick Williamson

11/08/2025

1. Introduction – Victor 4
1.1 Purpose 4
1.2 Document Conventions 4
1.3 Intended Audience and Reading Suggestions 4
1.3.1 Intended Audience 4
1.3.2 Reading Suggestions 5
1.4 Project Scope 5
1.4.1 Key Goals and Benefits 5
2. Overall Description 6
2.1 Product Features – Mark 6
2.2 User Classes and Characteristics – Atticus 6
2.2.1 User Class No.1 – The General User 6
2.2.2 User Class No.2 – The Security-Conscious User 7
2.2.3 User Class No.3 – The Technically-Inept User 7
2.3 Operating Environment – Atticus 8
2.3.1 Environment 8
2.3.3 Mobile Application Development 8
2.3.4 AI Development Model 8
2.3.5 Security and Permission Environment 9
2.4 Design and Implementation Constraints – Nick 9
2.4.1 Limiting Factors for Development 9
2.4.2 Platform Limitations 9
2.4.3 Data & Security Restriction 9
2.4.4 Malware Scope 9
2.4.5 Model execution constraints 10
3. System Features 10
3.1 Data Collection and Monitoring - Jacob 10
3.1.1 Description and Priority 10
3.1.2 Stimulus/Response Sequences 10
3.1.3 Functional Requirements 10
3.2 Feature Extraction & Selection – Mark 11
3.2.1 Description and Priority 11
3.2.2 Stimulus/Response Sequences 11
3.2.3 Functional Requirements 11
3.3 Machine Learning Model Training – Mark 11
3.3.1 Description and Priority 11
3.3.2 Stimulus/Response Sequences 11
3.3.3 Functional Requirements 12
3.4 Real-time Detection and Alerting - Jacob 12
3.4.1 Description and Priority 12
3.4.2 Stimulus/Response Sequences 12
3.4.3 Functional Requirements 12
4. External Interface Requirements 13
4.1 User Interfaces – Mark 13
4.2 Hardware Interfaces - Victor 13
4.2.1 Supported Hardware 13
4.2.1.1 Mobile Devices 13
4.2.2 Software-Hardware Interaction 13
4.2.3 Communication Protocols 13
4.3 Software Interfaces – Mark 13
5. Other Non-Functional Requirements – Tobias 14
5.1 Performance Requirements 14
5.2 Safety Requirements 14

5.3 Security Requirements 14
5.4 Software Quality Attributes 14
5.4.1 Availability 14
5.4.2 Modifiability 15
5.4.3 Reusability 15
5.4.4 Reliability 15
5.4.5 Usability 15
5.4.6 Legal 15
Group Attribution 16
Agile 16
Time Tracking 16
Sprint Review/Retrospective 16
Highlights 16
Minutes 16
Appendix 16
Appendix A: Minutes Drive 16

1. Introduction - Victor

1.1 Purpose

This document outlines the functional and non-functional requirements for the software component of the **AI-Driven Detection of Data-Exfiltration Malware on Mobile Devices** project. The system described is an application designed to detect information-stealing malware on mobile platforms through behavioural analysis powered by artificial intelligence.

This SRS focuses on the core detection subsystem of the application and covers its scope, intended users, high-level functionality, and constraints. The document is centred around the software responsible for:

- Capturing and processing behavioural data from mobile applications,
- Identifying features that represent normal vs. malicious activity,
- Training and evaluating AI/ML models for behavioural classification,
- Deploying a simplified prototype capable of demonstrating real-time or batch-based malware detection based on app behaviour.

This document does not cover full-scale mobile integration, app store distribution, or advanced UI/UX design. Instead, it focuses on the backend functionality and prototype demonstration of detection capabilities.

This is the first version (v1.0) of the SRS and applies to the initial development cycle spanning two academic semesters. Updates may be issued in future iterations based on changes in scope, data availability, or evaluation criteria.

1.2 Document Conventions

This Software Requirements Specification (SRS) follows the IEEE 830 standard for documenting software requirements. The following typographical and formatting conventions have been used throughout this document to maintain clarity and consistency:

- **Bold text** is used to highlight section titles and key words.
- Requirements are uniquely identified using the format [**REQ-#**], where "#" is a sequential identifier (e.g., REQ-1, REQ-2).
- Functional and non-functional requirements are separated and clearly labeled under their respective categories.

All diagrams and models follow the Unified Modeling Language (UML) standards where applicable.

1.3 Intended Audience and Reading Suggestions

This Software Requirements Specification (SRS) is intended for all stakeholders involved in the development, validation, and future enhancement of the **AI-Driven Detection of Data-Exfiltration Malware on Mobile Devices** project.

1.3.1 Intended Audience:

- **Project Supervisor (Mr. Keven Rashleigh), Unit Coordinator (Dr. Hannes Herrmann), Client (APROF Reza Ryan):**
- To evaluate the project scope, objectives, and progress against the capstone expectations.

- **Developers / Team Members / Testers (Victor Gan, Nick Williamson, Mark Pham, Atticus Kong, Tobias Jun, Jacob Sepkus):**
- To implement the behavioural analysis framework, machine learning models, and prototype application based on clearly defined requirements
- To follow the system design and development strategy as documented throughout the SRS.
- To design appropriate test cases that validate whether the system meets its functional and non-functional requirements.
- **Future Developers or Researchers:**
- To use this document as a baseline for further improvements, feature additions, or similar research efforts in mobile malware detection.

1.3.2 Reading Suggestions:

- Readers unfamiliar with the project should begin with Section 1 – Introduction, including the Purpose, Document Conventions, and Scope, to gain a high-level understanding.
- Those involved in the technical implementation (e.g., developers and data scientists) should focus on Section 3 – Specific Requirements, which outlines the functional and non-functional requirements, datasets, and system behaviour.
- Readers involved in project planning or evaluation (e.g., supervisors) should review Section 2 – Overall Description to understand the project's objectives, constraints, and system environment.
- Testers should refer primarily to the detailed functional requirements and validation criteria in Section 3, as well as any testing considerations mentioned in later appendices or supporting documents.

This document is organised to support both sequential reading and targeted reference, depending on the reader's role and interest.

1.4 Project Scope

The software specified in this document is a prototype AI-based detection system designed to identify data-exfiltration malware (commonly known as *infostealers*) on mobile platforms such as smartphones and tablets. The core objective is to develop a system capable of behavioural analysis of mobile applications to distinguish between normal (benign) and malicious app activity, particularly those attempting to steal sensitive user information.

The prototype system will use machine learning techniques trained on behavioural features (e.g., data access patterns, network usage, permission access) to detect malware. The solution focuses on early detection of threats by learning what typical app behaviour looks like and flagging deviations that may indicate malicious intent.

1.4.1 Key Goals and Benefits:

- **Enhance mobile security** through intelligent threat detection without relying solely on signature-based methods.
- **Demonstrate AI-driven behavioural analysis** as a viable solution for detecting modern mobile malware variants.
- **Support user privacy and data protection** by identifying suspicious apps that may compromise personal or confidential information.
- **Contribute to academic and research communities** through the development of a working prototype, a curated dataset, and experimental results.
- **Lay the foundation** for future integration into mobile antivirus solutions or enterprise mobile device management (MDM) tools.

This prototype supports broader cybersecurity goals by leveraging **artificial intelligence** to improve threat detection accuracy in dynamic and evolving threat environments, especially where traditional detection methods are ineffective.

2. Overall Description

2.1 Product Features – Mark

The system will contain the following key features for detecting and responding to data-exfiltration malware on Android devices:

1. **Data Collection & Monitoring**: Background collection of app behavioural data with minimal performance impact and user-controlled start/stop.
2. **Feature Extraction & Selection**: Automated identification of high-ranking behavioural indicators, and removal of irrelevant features to optimise efficiency.
3. **Machine Learning Classification**: Trained models which classify behaviours as benign or malicious.
4. **Real-Time Detection & Alerts**: Identification of suspicious activity, user notifications with details, and user options to deal with it.
5. **Secure Data Management**: Local storage of encrypted logs and results.
6. **Performance Targets**: Low CPU/memory usage, minimal battery drain, fast and accurate detection, and low false-positive rates.

2.2 User Classes and Characteristics – Atticus

This section identifies the different user classes that we anticipate will use this product and their user characteristics including educational level, technical expertise & knowledge, user behavior, and level of importance.

2.2.1 User Class No.1 – The General User

Description:

The typical or default user, representing the most common demographic, spanning a wide range of ages due to the ubiquity of mobile phones.

Educational Level:

Primary school up to university. (Everyone has a mobile phone)

Technical Expertise & Knowledge:

- Low to high expertise with a mobile phone and general app usage.
- Little to no experience with internet browsing.
- Little to no experience with data security.
- Little to no experience with cybersecurity.

User Behaviour:

- Users expect to be able to use the app without technical knowledge.
- Users will expect the app to function with minimal configuration.
- Users will interact only with simple GUI elements such as toggling the background process, receiving notifications, and reading app scan results.

Level of Importance:

Highest priority, this is the most important user class to satisfy as it is the majority of the user base.

2.2.2 User Class No.2 – The Security-Conscious User

Description:

A technically inclined user with an interest or knowledge in mobile security, privacy, and malware prevention.

Educational Level:

Tertiary-level education or self-taught in cybersecurity.

Technical Expertise & Knowledge:

- Moderate to high expertise with mobile operating systems, application behavior, and system settings.
- Moderate to High awareness of data privacy, permissions, and network security.
- May understand AI or behavioural analysis concepts at a basic level.

User Behaviour:

- Users may desire detailed insights into how apps behave and how the AI system makes decisions.
- Users may expect transparency, such as logs, data access histories, or permission summaries.
- Users will still expect a smooth and responsive app but may explore advanced settings if available.

Level of Importance:

This is the second most important user class, this user class is likely to use the app the most and require the most features to be satisfied.

2.2.3 User Class No.3 – The Technically-Inept User

Description:

A user with very limited experience using mobile applications or technology may

struggle with digital interfaces or understanding app permissions.

Educational Level:

Primary or early secondary education and elderly users or individuals with limited

access to formal technology training.

Technical Expertise & Knowledge:

- Very low expertise with mobile phones and general app usage.
- Unfamiliar with data privacy, cybersecurity, and how applications function.
- May be overwhelmed by technical language or multi-step processes.

User Behaviour:

- Users expect the app to work easily after installation, with little or no input required.
- Users may ignore or misunderstand notifications or warnings if they are not simple and direct.
- Users will likely only interact with large, obvious interface elements and avoid optional settings or advanced menus.

Level of Importance:

This will be the least important user class to satisfy, as they are both not common of a user base and not technically inept, catering the app to this user would likely diminish the overall quality of the app for all users.

2.3 Operating Environment – Atticus

This section outlines the hardware, software, and development environment for the system. The application will run on modern Android devices using Kotlin and Android Studio, with AI components developed in Python. It must operate efficiently in the background, comply with Android security models, and avoid interfering with other mobile apps.

2.3.1 Environment

The application will initially be designed for Android 11 (API level 30) and above, ensuring it works on a wide range of commonly used devices and Android emulators.

2.3.2 Hardware platform

Development and testing will be carried out on a Samsung Galaxy S21 FE, which will serve as the baseline for minimum supported specifications.

Baseline hardware requirements:

- Processor: Qualcomm Snapdragon 888 (Octa-core)
- RAM: 6GB
- Storage: 128GB internal (minimum 1GB free for application use)
- Display: 6.4" FHD+ AMOLED, 120Hz refresh rate
- Connectivity: Wi-Fi 6, 4G/5G, Bluetooth 5.0
- Operating System: Android 11 or later

2.3.3 Mobile Application Development

The mobile application will be developed for the Android platform, using Kotlin as the primary programming language within Android Studio as the development environment.

Frameworks & Libraries:

Potential options include the Android SDK, selected Jetpack libraries for lifecycle management and background task handling, and a suitable machine learning inference framework (e.g., TensorFlow Lite, ONNX Runtime Mobile, or similar). The final choice will be determined after evaluating compatibility, performance, and ease of integration.

2.3.4 AI Development Model

For the AI component we will consider multiple models such as random forests and neural networks and decide on which models to use and whether to use a single model or a combination of two with the exact libraries and frameworks to be determined during the design and prototyping stage.

Libraries:

Possible options include traditional machine learning libraries (e.g., Scikit-learn, XGBoost) or deep learning frameworks (e.g., TensorFlow, PyTorch), supported by data processing libraries such as Pandas and NumPy.

Implementation:

Models will be trained externally and converted for on-device inference using an appropriate deployment framework (e.g., TensorFlow Lite, ONNX Runtime Mobile, or similar). The selection will be based on performance, resource usage, and ease of deployment on Android devices.

Backend / Storage:

If a backend is required, possible options include Firebase, AWS, or a local server to store datasets or provide model updates. The approach will be finalised after testing local-only operation versus server-assisted functionality.

2.3.5 Security and Permission Environment

The application will adhere to Android's standard permission and security models, operating without the need for root access. Required permissions (e.g., for monitoring network activity or accessing system data) will be requested with clear justification to the user.

Security Measures:

Potential measures include ensuring all processing is local, using encrypted communication channels if data transfer is required, and considering privacy-preserving methods such as federated learning or explainable AI for transparency. The exact methods will be chosen after further design and threat analysis.

2.4 Design and Implementation Constraints – Nick

2.4.1 Limiting Factors for Development

As we plan to develop our malware detection system there are a few development factors to consider, we will develop for an android based mobile device, using Kotlin and we develop on android studio. iOS is not feasible for development due to Apple's requirement of developing on a macOS device using SWIFT, as not all developers involved in the project have a native device; we are limited to Android development. Time will also be the largest development factor as the team is working within a six month period with ten hours assigned weekly.

2.4.2 Platform Limitations

We must test and implement within the given limitations that our chosen development will allow us. Understanding that we will be based on Android we will only be able to test and design for the devices we have access to and will have to cater our app around the limits of said device. The use of emulators may help to mimic further devices but guarantees can only be made for devices that have been properly implemented and tested.

2.4.3 Data & Security Restriction

Due to the requirements of the universities ethical and security concerns we will be limited in the range of data we can test and collect. We are required to not involve any real data when training and testing data meaning we will be limited to an appropriate source of test data. Our testing must also take place in a controlled environment to ensure that we are not imposing a real threat as we propagate malware.

2.4.4 Malware Scope

The project description didn't define the specific types and behaviours of infostealer malware thus we will define later our specific desired targets. This is due to our limited resources constraining us to target a specific genre of malware in order to do our best to guarantee effectiveness of our AI detection.

2.4.5 Model execution constraints

The desired design of the app is to take place in the background of the device and not require consistent intervention from the user. This is while we have to place limits on the AI to ensure it will be lightweight and efficient in running to allow the user to still use the device with standard expectations and not disrupt day to day use.

2.5.6 Corporate and regulatory policies

Must comply with university ethical policies which requires us not to collect any personal data or test on any public devices.

3. System Features

3.1 Data Collection and Monitoring – Jacob

3.1.1 Description and Priority

This feature involves continuously monitoring the behaviour of mobile apps to detect any suspicious actions related to data-exfiltration malware. It forms the backbone of the malware detection system, enabling real-time detection and analysis.

- **Priority:** High

3.1.2 Stimulus/Response Sequences

- **Stimulus:** User installs an app on the mobile device.
- **Response:** The system starts monitoring the app's behaviour (e.g. API usage, network traffic, file I/O).
- **Stimulus:** The app performs a suspicious action, such as accessing sensitive data or making unusual network requests.
- **Response:** The system logs the behaviour and prepares the data for further analysis.
- **Stimulus:** User toggles monitoring to be turned off.
- **Response:** The system will stop monitoring app behaviour.
- **Stimulus:** System resources are being used by another process on the device.
- **Response:** The program limits resource consumption so that device remains usable.

3.1.3 Functional Requirements

- **REQ-1:** The system will collect data from mobile applications, including API calls, permissions, network activity, and file access patterns.
- **REQ-2:** The system will continuously monitor the app's interactions with the device during its execution.
- **REQ-3:** All collected data will be securely logged for future analysis, ensuring traceability of app behaviour.
- **REQ-4:** The system will provide an option to pause or stop monitoring when required by the user, ensuring that no unnecessary data is collected.
- **REQ-5:** Monitoring will not affect the overall performance or usability of the device.

3.2 Feature Extraction & Selection – Mark

3.2.1 Description and Priority

This feature processes behavioural and static data collected during app execution to select the most indicative signs of data-exfiltration malware. It will incorporate a hybrid approach, using lightweight static analysis (permissions, API usage) and dynamic analysis (network activity, API calls sequences, file I/O) to provide resilience against obfuscation. The features will be ranked by importance via machine learning model/s to optimise computational efficiency.

- **Priority:** High

3.2.2 Stimulus/Response Sequences

- **Stimulus:** The monitoring module sends raw app behaviour logs.
- **Response:** The system processes the data into relevant features such as network traffic patterns, API calls and permission usage.
- **Stimulus:** A new dataset is received.
- **Response:** The system uses ranks features by discriminative power, discarding redundant ones and updates the feature set.

3.2.3 Functional Requirements

- **REQ-6:** The system will extract a set of static and dynamic features, including:
  - Mean/variance of network packet sizes, flow durations, upstream/downstream packet ratios
  - Frequency of sensitive API calls
  - Unusual permission combinations
- **REQ-7:** The system will perform feature ranking, preferably using deep-learning based methods, to prioritise the most discriminative features for classification.
- **REQ-8:** The system will discard irrelevant features to maximise computational efficiency without losing accuracy.
- **REQ-9:** The system will aim to be resistant to obfuscation techniques.

3.3 Machine Learning Model Training – Mark

3.3.1 Description and Priority

This feature uses the optimised feature set to train machine learning models capable of classifying app behaviour as benign or malicious. A hybrid model will be adopted – Random Forest and SVM for efficiency and robustness, with optional deep-learning. The models will support incremental learning to adapt to new threats, improving long-term effectiveness against evolving malware.

- **Priority**: High

3.3.2 Stimulus/Response Sequences

- **Stimulus:** A labelled dataset of benign and malicious app behaviour is available.
- **Response:** The system trains classification model/s using the dataset's optimised feature set.
- **Stimulus:** New labelled data is added after initial training.

- **Response:** The system updates the model incrementally, continuously improving whilst retaining older knowledge.

3.3.3 Functional Requirements

- **REQ-10:** The system will train classification models using labelled behavioural datasets, including Android-specific datasets (e.g. Drebin, CICAndMal2017) tailored to data-exfiltration threats.
- **REQ-11:** The system will implement algorithm/s, potentially including:
  - Random Forest and SVM for non-linear feature interactions.
  - LSTM or CNN-LSTM for capturing temporal patterns in malicious behaviour.
- **REQ-12:** The system shall perform cross-validation during training to ensure generalisability and mitigate overfitting to a single dataset.
- **REQ-13:** The system shall support incremental model updates (e.g. online learning) to adapt to new malware behaviours without retraining from scratch.
- **REQ-14:** The system shall store trained models locally in a secure format and reload them for future use.

3.4 Real-time Detection and Alerting - Jacob

3.4.1 Description and Priority

This feature enables the system to detect malicious behaviour during the execution of mobile apps and trigger real-time alerts if suspicious activity is detected. It is crucial for the system to provide timely feedback to users or security teams.

- **Priority:** High

3.4.2 Stimulus/Response Sequences

- **Stimulus:** The system monitors app behavior in real-time.
- **Response:** If suspicious behavior is detected (e.g., abnormal network traffic), the system triggers an alert.
- **Stimulus:** A suspicious activity (e.g., abnormal network traffic or API call pattern) is detected.
- **Response:** The system generates an alert and logs the event for further investigation.
- **Stimulus:** The user taps on the alert notification.
- **Response:** The application opens to and shows the specific details of the alert.
- **Stimulus:** The user takes action on the alert.
- **Response:** The notification will be cleared.

3.4.3 Functional Requirements

- **REQ-15:** The system will monitor app behavior in real-time, including network activity, API calls, and file I/O operations.
- **REQ-16:** The system will detect suspicious behavior indicative of data exfiltration, such as abnormal network connections or excessive data requests.
- **REQ-17:** The system will generate real-time alerts when malicious behaviour is detected, including a description of the suspicious activity.

- **REQ-18:** The system will allow users to take action based on the alert, such as quarantining the app or terminating its execution.
- **REQ-19:** The notification will be cleared after the user takes action, however, details and history of the alert will still remain in the application.

4. External Interface Requirements

4.1 User Interfaces – Mark

The system will be an Android-optimised graphical user interface designed for simplicity

4.2 Hardware Interfaces - Victor

4.2.1 Supported Hardware

4.2.1.1 Mobile Devices (Android)

- No physical mobile device is required for the current prototype.
- Mobile behaviour data will be **simulated or sourced** from public datasets to emulate interactions that occur on Android devices.

4.2.2 Software-Hardware Interaction

- The software will **not** directly interface with mobile hardware components such as sensors, radio modules, or OS-level APIs.
- Behavioural data used for model training and testing (e.g., API calls, permission usage, network activity) will be in a pre-recorded format (CSV/JSON) and processed entirely on the host system.
- If Android Emulator or sandboxing environments are used to capture dynamic behaviour, they will be set up independently and are not integrated into the prototype.

4.2.3 Communication Protocols

- No direct communication protocols (e.g., USB, Bluetooth, or mobile OS APIs) are used in the current implementation.
- Internal file system access is used for reading input datasets and saving detection results.
- In future iterations, the system could be extended to support real-time behavioural monitoring on physical or virtual mobile devices via:
  - **Android Debug Bridge (ADB)** for device communication and log retrieval.
  - **System call tracing tools** to monitor app-level interactions with the OS.
  - **Custom logging agents** embedded within test mobile environments to collect behavioural data in real time.

4.3 Software Interfaces – Mark

The system will interact with a range of software components and libraries to support data ingestion, processing, machine learning model execution, and reporting. It will be compatible with Windows 10/11, macOS, and Linux for development and testing, requiring a Python runtime environment. Core libraries will include scikit-learn for machine learning algorithms, with the option to generate PDF or HTML reports for user review. The development team will use Bitbucket (Git-based) for collaborative code management and CI/CD integration.

5. Other Non-Functional Requirements – Tobias

5.1 Performance Requirements

- **REQ-1:** The system will aim for a minimal response time when there is user input (button pressing, navigation etc.) for a desirable user experience
- **REQ-2:** The app will aim to have a maximised malware protection rate
- **REQ-3:** The system will work aim to faster than malware can cause damage for the app to be even viable for use
- **REQ-4:** The system will aim to be able to scan the device quickly and efficiently without using much of the CPU so that the app can run without affecting the user
- **REQ-5:** The AI model used to scan devices will aim to be efficient with a maximised accuracy so that there is a less likelihood of malware getting through
- **REQ-6:** The app will aim to have battery drainage that is not invasive so it does not affect users

5.2 Safety Requirements

- **REQ-7:** The program will aim to have limited access to data as certain bugs/events/issues can arise causing the program to remove important data thinking it was malware.
- **REQ-8:** The program will use permissions within boundaries and not abuse these permissions so the app will have no capability in harming the device.
- **REQ-9:** The system will quarantine suspicious files instead of deleting them to ensure important files are not deleted.

5.3 Security Requirements

- **REQ-10:** Entering the app will require high security (can enter wrong passwords only twice in a period of 24 hours, certain password arrangements etc.) so third parties cannot interact with the app
- **REQ-11:** The system will aim to make sure all data is encrypted so the users' data is secure
- **REQ-12:** The system will aim to secure storage internally on the app and if external will aim to make sure it is not sensitive data so sensitive data is not able to be leaked
- **REQ-13:** The system will have an automatic session timeout after an appropriate amount of time of inactivity to make sure there is not any unwanted access to the app for a third-party

5.4 Software Quality Attributes

5.4.1 Availability

This project uses the platform of an app so the platforms chosen were Android.

- **REQ-14:** The system will aim to be able to work on most modern android phones (Android 11+) to stay relevant as well as be helpful to those with older models

5.4.2 Modifiability

The app needs to be able to be easily modified for changes such as new modern technology, more efficient code, moving to another platform etc.

- **REQ-15:** The app's code will be able to be easily modified for future updates with clean code so that updates can be produced quicker

- **REQ-16:** The app's code will be easy to understand to convert to other platforms so that plans to move to different platforms are available
- **REQ-17:** The app will be able to be easily modified for future platform updates so that the app can stay relevant

5.4.3 Reusability

The software needs reusable components for efficiency.

- **REQ-18:** The AI model will be reusable across different platforms rather than creating a new one for more efficiency

5.4.4 Reliability

The app needs to be reliable and have a failsafe for potential risks.

- **REQ-19:** The app will aim to have a high percentage of uptime with a continual updating antivirus system to make sure the app is protecting the device
- **REQ-20:** If the system fails the system will aim to make a quick recovery to minimise potential damages of malware through being able to detect failures and notifying the team fast
- **REQ-21:** The system will aim to have a minimal amount of false-positives
- **REQ-22:** The system will log all errors and important operations for future analysis
- **REQ-23:** The system will attempt to retry failed scans or updates and will notify the user if still unsuccessful

5.4.5 Usability

The app needs to provide the user with a positive experience when using the app.

- **REQ-24:** The account making process and anti-malware program setup will be able to be set up in a convenient amount of time and as this would be the only necessary interaction with the app, it will be beneficial to users' conveniences.
- **REQ-25:** The system will aim to do everything else internally without need for user interaction so the user can have peace of mind without manual labour.
- **REQ-26:** The settings options will be easy to understand to personalise the system for the user so the user can provide input in how they want the app to run.

5.4.6 Legal

The app should adhere to laws and work in the legal green area.

- **REQ-27:** The system will respect data privacy through encryption or deletion after use to make sure the app does not break any privacy laws
- **REQ-28:** The app will go through the right procedure when asking for permissions so that it is not installed in devices that are shared but owned by another entity (university property, company property etc.)
- **REQ-29:** The app will provide a privacy policy and terms of use at signup

Group Attribution

Team members' names were written to the sections they completed.

Agile

Kanban Board

Time Tracking

Tobias's Toggl Hours: 18 hours, 57 minutes

Mark's Toggl Hours: 18 hours, 46 minutes

Jacob's Toggl Hours: 17 Hours, 22 Minutes

Victor's Toggl Hours: 14 Hours 47 Minutes

Nick's Toggl Hours: 15 Hours, 43 Minutes

Atticus' Toggl Hours: 17 Hours, 40 Minutes

Sprint Review/Retrospective

In this sprint, we successfully completed our first draft of the SRS. Our team worked well together, maintaining consistent communication and resolving issues quickly. Moving forward, we will aim to improve our planning process and task allocation to prevent issues such as redundant work. We will also aim to allocate effort to tasks for an easier understanding of task priorities. Next sprint, we will decide on and apply a method of task and effort allocation, and continue to build on our strength in team collaboration.

Highlights

Minutes

Tobias Jun has been taking the minutes for the meetings.

Minutes (Appendix A)

Appendix

Appendix A: Minutes Drive

https://drive.google.com/drive/folders/1GBQ5VD8MR4fO3QbBsUxS-l5MNwFsVdu9?usp=drive_link